



भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Indian Institute of Technology Hyderabad

Analyzing MLIR GPU Dialect for Neural Network Workloads and Proposing Optimizations

Contents

1	Problem Statement	2
2	Survey Report: MLIR and ResNet-50	3
2.1	MLIR Dialect Stack Overview	3
2.2	Background: ResNet50 Workload	6
3	Cutlass Optimization in Bufferized Linalg Dialect	7
3.1	Linalg Pattern Matching and Dependency Analysis Passes	7
3.2	CUTLASS-Based Convolution Lowering Pass	9
3.3	A look on Transformed IR -	10
3.4	Experimental Results	11
4	PyTorch Compilation and CUDA Graph Integration	12
4.1	PyTorch and Compilation Framework	12
4.2	CUDA Graphs in PyTorch	12
4.3	Challenges in Direct CUDA Graph Deployment	12
4.4	PyGraph: Robust Compiler Support for CUDA Graphs in PyTorch	12
5	Analysis of GPU Dialect IR	13
5.1	Optimizations Proposed on GPU Dialect	13
6	Future Directions	14
7	References	14

1 Problem Statement

- Deep Neural Networks such as ResNet-50 involve heavy tensor computations and typically execute on GPUs.
- MLIR lowers machine learning workloads through multiple dialects, starting from the Torch dialect and progressing through Linalg, Affine/SCF, and finally the GPU dialect.
- The Torch dialect captures high-level tensor semantics, while Linalg and Affine/SCF introduce structured loop-based representations suitable for optimization.
- At the GPU dialect level, the IR reflects the CUDA execution model with threads, blocks, shared memory, and synchronization.
- However, even after lowering to the GPU dialect, the generated IR may still contain inefficiencies such as:
 - redundant global memory accesses,
 - suboptimal kernel launch parameters,
 - unnecessary synchronization,
 - lack of shared-memory reuse.
- These inefficiencies reduce GPU occupancy and overall performance.
- The goal of this project is to analyze the GPU dialect IR of ResNet-50, identify where the compiler fails to optimize, and propose improvements such as memory promotion, kernel fusion, and warp specialization.

2 Survey Report: MLIR and ResNet-50

2.1 MLIR Dialect Stack Overview

MLIR is built as a multi-dialect system, where each dialect represents the computation at a different level of abstraction. Instead of lowering directly from a high-level model to GPU instructions, MLIR progressively transforms the IR across a stack of specialized dialects. This enables the compiler to apply the right optimizations at the right stage of the compilation pipeline.

- SSA-based compiler IRs provide multiple layers of abstraction suitable for building Domain Specific Languages (DSLs).
- MLIR offloads the target machine code generation phase to the LLVM compiler infrastructure.
- The fundamental entity in MLIR is the **Operation**; in MLIR, everything is an operation, including function definitions.
- An operation takes arguments and contains one or more **regions**.
- Each region consists of one or more **blocks** that define control flow.
- A block itself contains operations, allowing recursive nesting of operations within blocks.
- For example:
 - A `torch.conv` call in the PyTorch DSL
 - Then becomes `torch.aten.convolution` in Torch-MLIR,
 - Then to `linalg.conv_2d_nchw_fchw` in the Linalg dialect,
 - and eventually becomes nested loop constructs in the Affine dialect, etc.

Torch Dialect

- Models PyTorch programs with high-level tensor operators (e.g., `aten.matmul`, `aten.conv`, `aten.relu`).
- Preserves semantics such as broadcasting and dynamic shapes.
- Performs frontend optimizations like constant folding and dead-code elimination.
- No explicit loops or hardware details at this level.

Linalg Dialect

- Represents computations using structured linear algebra operators (`linalg.matmul`, `linalg.conv`, `linalg.generic`).
- Exposes iteration spaces and indexing maps explicitly.
- Enables optimizations like tiling, vectorization, fusion, and memory promotion.
- Serves as the key stage for performance-critical transformations before lowering to loops.

The following is the mlir snippet of linalg dialect from resnet50.

```
%1 = linalg.fill ins(%cst_0 : f32) outs(%0 : tensor<1x64x112x112xf32>)
-> tensor<1x64x112x112xf32>
%2 = linalg.conv_2d_nchw_fchw {dilations = dense<1> : vector<2xi64>, strides = dense<2> :
vector<2xi64>} ins(%padded, %cst : tensor<1x3x230x230xf32>, tensor<64x3x7x7xf32>) outs(%1 :
tensor<1x64x112x112xf32>) -> tensor<1x64x112x112xf32>
```

Affine Dialect

- Represents loops and memory access using affine expressions.
- Allows dependence analysis and loop-level optimizations such as interchange, tiling, fusion, and parallelization.
- Suitable for workloads with static shapes and predictable memory access.

SCF Dialect

- Handles structured control flow using `scf.for`, `scf.if`, and `scf.parallel`.
- More general than Affine — supports dynamic shapes and non-affine indexing.
- Common lowering target for Linalg before generating explicit loop nests.
- Provides a bridge to GPU code generation.

```
%c0_7 = arith.constant 0 : index
%c1_8 = arith.constant 1 : index
%c1_9 = arith.constant 1 : index
scf.parallel (%arg160) = (%c0_7) to (%c1_8) step (%c1_9) {
  %c0_1913 = arith.constant 0 : index
  %c64_1914 = arith.constant 64 : index
  %c1_1915 = arith.constant 1 : index
  scf.parallel (%arg161) = (%c0_1913) to (%c64_1914) step (%c1_1915) {
    %c0_1916 = arith.constant 0 : index
    %c112 = arith.constant 112 : index
    %c1_1917 = arith.constant 1 : index
    scf.parallel (%arg162) = (%c0_1916) to (%c112) step (%c1_1917) {
      %c0_1918 = arith.constant 0 : index
      %c112_1919 = arith.constant 112 : index
      %c1_1920 = arith.constant 1 : index
      scf.parallel (%arg163) = (%c0_1918) to (%c112_1919) step (%c1_1920) {
        memref.store %cst_3, %alloc_6[%arg160, %arg161, %arg162, %arg163] :
          memref<1x64x112x112xf32>
      }
    }
  }
}
```

When lowering this IR to the GPU dialect, the outermost loops are typically mapped to gpu **thread blocks**, while the next level of loops is mapped to **threads within a block**. Specifically:

- The first outer loop (`%arg160 = 0 to 1`), which iterates over the batch dimension, is assigned to thread blocks.
- The second outer loop (`%arg161 = 0 to 64`), which iterates over output channels, is assigned to individual threads within each block.

This mapping exposes parallelism at the block and thread level, allowing the GPU to execute multiple convolution computations concurrently. However, having many nested loops can create overhead due to loop management and redundant index calculations. One optimization strategy is **loop fusion** or merging, where two or more loops at the outer levels are combined into a single loop before mapping to thread blocks and threads.

GPU Dialect

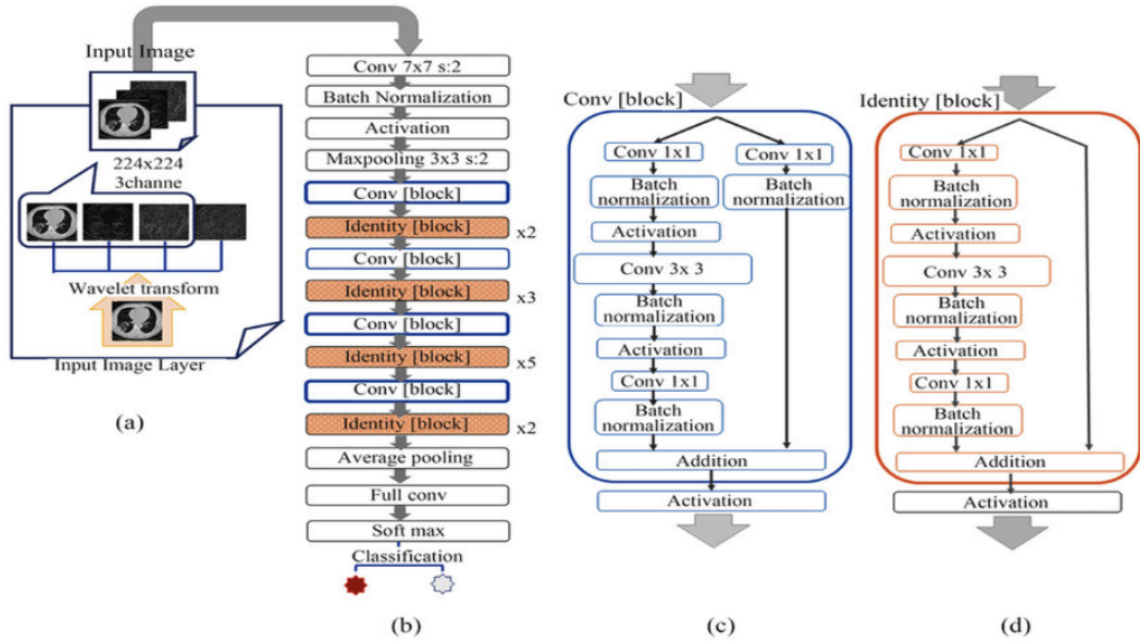
- Represents GPU execution using `gpu.launch`, `gpu.blockId`, `gpu.threadId`, etc.
- Makes thread hierarchy, shared memory, and synchronization explicit.
- Enables optimizations such as shared-memory reuse, synchronization removal, and kernel launch tuning.
- Acts as the final IR stage before lowering to vendor-specific backends like NVVM for NVIDIA GPUs.

The following mlir GPU dialect snippet is of fill/memset operation from linalg.

```
gpu.launch blocks(%arg160, %arg161, %arg162) in (%arg166 = %163, %arg167 = %c1_13, %arg168 =  
%c1_13) threads(%arg163, %arg164, %arg165) in (%arg169 = %164, %arg170 = %c1_13, %arg171 =  
%c1_13) {  
  %1005 = affine.apply #map1(%arg160)[%c1_12, %c0_10]  
  %c0_3493 = arith.constant 0 : index  
  %c64_3494 = arith.constant 64 : index  
  %c1_3495 = arith.constant 1 : index  
  %c1_3496 = arith.constant 1 : index  
  %c0_3497 = arith.constant 0 : index  
  %1006 = affine.apply #map1(%arg163)[%c1_3496, %c0_3497]  
  %c0_3498 = arith.constant 0 : index  
  %c112 = arith.constant 112 : index  
  %c1_3499 = arith.constant 1 : index  
  scf.for %arg172 = %c0_3498 to %c112 step %c1_3499 {  
    %c0_3500 = arith.constant 0 : index  
    %c112_3501 = arith.constant 112 : index  
    %c1_3502 = arith.constant 1 : index  
    scf.for %arg173 = %c0_3500 to %c112_3501 step %c1_3502 {  
      memref.store %cst_3, %alloc_9[%1005, %1006, %arg172, %arg173] : memref<1x64x112x112xf32>  
    }  
  }  
  gpu.terminator  
} {SCFToGPU_visited}
```

2.2 Background: ResNet50 Workload

- ResNet50 (Residual Network-50) is a deep convolutional neural network with 50 layers designed for large-scale image classification tasks such as ImageNet.
- The architecture consists of convolution, batch normalization, ReLU, and pooling layers organized into four main stages, each containing multiple residual blocks.
- Each residual block performs identity mapping by adding the input directly to the output of stacked convolution layers, forming shortcut (skip) connections.
- The bottleneck structure (1×1 , 3×3 , 1×1 convolutions) reduces computation while preserving representational power.
- ResNet50 takes a 224×224 RGB image as input and outputs a 1000-dimensional prediction vector using a fully connected layer followed by softmax.
- The model is widely used as a strong feature extractor and as a baseline for transfer learning and general computer vision applications.
- Computationally, ResNet50 heavily relies on matrix multiplications and convolutions, making it a demanding GPU workload and a key candidate for performance optimization studies.
- Its repetitive hierarchical structure makes it suitable for MLIR-based analysis, optimization, and dialect lowering across the compilation pipeline.



<https://wisdomml.in/understanding-resnet-50-in-depth-architecture-skip-connections-and-advantages-over-other-networks/>

Figure 1: Compiler Engine Overview

3 Cutlass Optimization in Bufferized Linalg Dialect

3.1 Linalg Pattern Matching and Dependency Analysis Passes

Linalg Operation Sequence Analysis Pass

- Collects all `linalg::LinalgOp` instances within a function.
- Iterates over the collected list and prints for each operation:
 - the current Linalg operation,
 - the previous two operations (if they exist),
 - the next two operations (if they exist).
- Helps to understand the execution order of Linalg operations in the IR.
- Useful for identifying opportunities for:
 - kernel fusion,
 - grouping repeated computation patterns inside ResNet50 blocks.

```
Current Linalg Operation: linalg.generic
Previous two operations:
- linalg.generic
- linalg.generic
Next two operations:
- linalg.conv_2d_nchw_fchw
- linalg.fill

Current Linalg Operation: linalg.conv_2d_nchw_fchw
Previous two operations:
- linalg.generic
- linalg.generic
Next two operations:
- linalg.fill
- linalg.pooling_nchw_max

Current Linalg Operation: linalg.fill
Previous two operations:
- linalg.generic
- linalg.conv_2d_nchw_fchw
Next two operations:
- linalg.pooling_nchw_max
- linalg.fill
```

Figure 2: Output of Operation Sequence Analysis Pass

Linalg Dependency Analysis Pass

- Constructs a dependency map using each Linalg operation's output (LHS value) and its input tensors (DPS inputs).
- For every Linalg operation, the pass prints:
 - the result tensor produced,
 - the input operands consumed,
 - all Linalg operations that compute values used by the current operation.
- Reveals producer-consumer relationships between Linalg operations.
- Enables systematic detection of optimization opportunities, including:
 - legal kernel fusion,
 - asynchronous streaming of independent operators,
 - Promotion for tensors reused in all operations.
- Provides a dataflow-aware view of Linalg computation rather than purely structural ordering.

```
LHS Value: %525
Associated LinalgOp: linalg.pooling_nchw_sum
Inputs: %521, %524,
Dependent on: linalg.generic(%521),

LHS Value: %526
Associated LinalgOp: linalg.generic
Inputs: %525,
Dependent on: linalg.pooling_nchw_sum(%525),

LHS Value: %transposed
Associated LinalgOp: linalg.transpose
Inputs: %cst_163,
Direct Dependencies: None

LHS Value: %529
Associated LinalgOp: linalg.fill
Inputs: %cst_0,
Direct Dependencies: None

LHS Value: %530
Associated LinalgOp: linalg.matmul
Inputs: %collapsed, %transposed,
Dependent on: linalg.transpose(%transposed),

LHS Value: %531
Associated LinalgOp: linalg.generic
Inputs: %530, %cst_164,
Dependent on: linalg.matmul(%530),
```

Figure 3: Output of Operation Dependency Pass

3.2 CUTLASS-Based Convolution Lowering Pass

This pass targets the lowering of `linalg::Conv2DNchwFchwOp` operations to an explicit call to a CUTLASS-based GPU kernel. The transformation is applied on the bufferized Linalg dialect and converts high-level convolution operators into custom device-callable kernels to leverage vendor-optimized GEMM-based implementations.

Objectives

- Remove the default lowering pipeline for `linalg.conv_2d` and instead redirect convolution execution to a CUTLASS kernel.
- Enable invocation of a hand-written, architecture-optimized CUDA kernel using `func.call`.
- Preserve compatibility with the existing bufferization and MemRef-based memory model used in downstream GPU lowering.

Pattern Matching Logic

- The pass registers a rewrite pattern that specifically matches `linalg::Conv2DNchwFchwOp`.
- The pattern is triggered only when the following conditions hold:
 - Inputs have either `memref` or `tensor` types.
- Upon a successful match, the convolution op is replaced with a device kernel call rather than further MLIR-based lowering.

Operand Extraction and Argument Formation

- Extracts the three data operands:
 - input activation
 - filter (weights)
 - output tensor
- Computes convolution-relevant shape parameters using `arith.constant` operations:
 - batch size (N), input channels (C), input height/width (H, W),
 - filter output channels (K), filter height/width (R, S),
 - output height/width (P, Q).
- Extracts **stride**, **dilation**, and **padding** information.
- All tensor arguments are cast to `unranked.memref<f32>` to ensure uniform kernel ABI compatibility.

Kernel Resolution and Creation

- A helper function `getOrCreateCutlassConvKernel()` ensures that the CUTLASS kernel function is present in the module symbol table.
- If not already defined, the pass inserts the following into the module:
 - A private `func.func` symbol named `"cutlass_conv2d_kernel"`.
 - A function type consisting of:
 - * three unranked memref arguments (output, input, filter),
 - * eighteen index-type parameters encoding shapes and convolution hyperparameters.
 - The attribute `llvm.emit_c_interface` enabling external CUDA linkage.

Final Rewrite

- The original `linalg::Conv2DNchwFchwOp` is erased from the IR.
- A `func.call` operation is inserted to invoke:

```
cutlass_conv2d_kernel(output, input, filter, shapes, strides, dilations, pads)
```

- No return value is expected; the call writes directly into the output buffer.

Benefits of the Transformation

- Enables deployment of vendor-optimized CUTLASS convolution primitives.

3.3 A look on Transformed IR -

The bufferized Linalg IR before transformation contains multiple independent Linalg operations operating on tensors and memrefs without. And when these operations get lowered to furthered dialect have unoptimized launching parameters for gpu dialect.

```
%subview = memref.subview %alloc[0, 0, 3, 3] [1, 3, 224, 224] [1, 1, 1, 1] : memref<1x3x230x230xf32> to memref<1x3x224x224xf32, strided<[158700, 52900, 230, 1], offset: 693>>
memref.copy %arg159, %subview : memref<1x3x224x224xf32, strided<[7, 7, 7, 7], offset: 7>> to memref<1x3x224x224xf32, strided<[158700, 52900, 230, 1], offset: 693>>
%alloc_4 = memref.alloc() {alignment = 64 : i64} : memref<1x64x112x112xf32>
linalg.fill ins(%cst : f32) outs(%alloc_4 : memref<1x64x112x112xf32>)
%alloc_5 = memref.alloc() {alignment = 64 : i64} : memref<1x64x112x112xf32>
memref.copy %alloc_4, %alloc_5 : memref<1x64x112x112xf32> to memref<1x64x112x112xf32>
linalg.conv_2d_nchw_fchw {dilations = dense<1> : vector<2xi64>, strides = dense<2> : vector<2xi64>} ins(%alloc, %0 : memref<1x3x230x230xf32>, memref<64x3x7x7xf32>) outs(%alloc_5 : memref<1x64x112x112xf32>)
%alloc_6 = memref.alloc() {alignment = 64 : i64} : memref<64xf32>
linalg.generic {indexing maps = [#map, #map], iterator_types = ["parallel"]} ins(%arg1 : memref<64xf32, strided<[?], offset: 7>>) outs(%alloc_6 : memref<64xf32>) {
  ^bb0(%in: f32, %out: f32):
    %i61 = arith.truncf %cst 2 : f64 to f32
    %i62 = arith.addf %in, %i61 : f32
    linalg.yield %i62 : f32
}
```

Figure 4: Bufferized Linalg Dialect before Transformation

After transformation, the bufferized Linalg IR exhibits to call optimized CUTLASS kernels.

```
%subview = memref.subview %alloc[0, 0, 3, 3] [1, 3, 224, 224] [1, 1, 1, 1] : memref<1x3x230x230xf32> to memref<1x3x224x224xf32, strided<[158700, 52900, 230, 1], offset: 693>>
memref.copy %arg159, %subview : memref<1x3x224x224xf32, strided<[7, 7, 7, 7], offset: 7>> to memref<1x3x224x224xf32, strided<[158700, 52900, 230, 1], offset: 693>>
%alloc_4 = memref.alloc() {alignment = 64 : i64} : memref<1x64x112x112xf32>
linalg.fill ins(%cst : f32) outs(%alloc_4 : memref<1x64x112x112xf32>)
%alloc_5 = memref.alloc() {alignment = 64 : i64} : memref<1x64x112x112xf32>
memref.copy %alloc_4, %alloc_5 : memref<1x64x112x112xf32> to memref<1x64x112x112xf32>
%cast = memref.cast %alloc_5 : memref<1x64x112x112xf32> to memref<xf32>
%cast_6 = memref.cast %alloc : memref<1x3x230x230xf32> to memref<xf32>
%cast_7 = memref.cast %0 : memref<64x3x7x7xf32> to memref<xf32>
call @cutlass_conv2d_kernel(%cast, %cast_6, %cast_7, %c1, %c3, %c230, %c230, %c64, %c7, %c7, %c112, %c112, %c2, %c2, %c3, %c3, %c1, %c1) : (memref<xf32>, memref<xf32>, memref<xf32>,
%alloc_8 = memref.alloc() {alignment = 64 : i64} : memref<64xf32>
linalg.generic {indexing maps = [#map, #map], iterator_types = ["parallel"]} ins(%arg1 : memref<64xf32, strided<[?], offset: 7>>) outs(%alloc_8 : memref<64xf32>) {
  ^bb0(%in: f32, %out: f32):
    %i61 = arith.truncf %cst 0 : f64 to f32
    %i62 = arith.addf %in, %i61 : f32
    linalg.yield %i62 : f32
}
```

Figure 5: Bufferized Linalg Dialect after Transformation

3.4 Experimental Results

- To evaluate the benefit of CUTLASS integration and calling in mlir, we isolated and benchmarked:
 1. The CUTLASS Conv2D template without MLIR involvement with dummy inputs and
 2. A CUDA Kernel equivalent to MLIR-generated convolution kernel (baseline) with similar inputs
- Both kernels were executed on an NVIDIA A100 GPU, and the mean execution time was measured.

Implementation	Avg Execution Time of 10 runs(ms)
CUTLASS Conv2D (Template)	0.108442
MLIR Conv (Baseline Kernel)	35.0116

We observed high improvement when using CUTLASS Conv2D compared to the MLIR-generated convolution kernel. We believe that on testing MLIR executable with Since ResNet-50 contains around **53 convolution layers**, replacing all MLIR-generated kernels with CUTLASS template calls has the potential to deliver a substantial end-to-end performance improvement.

4 PyTorch Compilation and CUDA Graph Integration

As a part of optimization we also read about the pytorch compilation and integration of CUDA Graph in it. We found the below summary of resources which we looked at :

4.1 PyTorch and Compilation Framework

PyTorch offers compiler-based optimizations through `torch.compile()`, which lowers Python code to efficient GPU kernels. PyTorch 2.0 introduces a modular compilation pipeline consisting of two major components:

- **Dynamo** – Extracts graph-level information from Python code.
- **Inductor** – Generates optimized kernels targeting CPUs and GPUs.

4.2 CUDA Graphs in PyTorch

CUDA Graphs are an NVIDIA feature designed to reduce the overhead of repeatedly launching GPU kernels from the CPU. This is particularly impactful for deep learning workloads, where layers are executed iteratively during training and inference.

- Repeated kernel launches in loops incur significant CPU-side cost.
- CUDA Graphs record a sequence of GPU kernels and replay them with minimal launch overhead.
- They are most beneficial when execution patterns are stable, as in deep learning workloads.

4.3 Challenges in Direct CUDA Graph Deployment

While CUDA Graphs offer large performance benefits, their deployment in PyTorch introduces practical difficulties:

- Not all kernels benefit equally from graph execution.
- Scalar parameters inside kernel arguments complicate replayability.
- Copying kernel parameters for each replay adds runtime overhead.

Therefore, a compiler-based approach is needed to selectively and intelligently integrate CUDA Graphs within PyTorch.

4.4 PyGraph: Robust Compiler Support for CUDA Graphs in PyTorch

PyGraph is designed to integrate CUDA Graphs into the PyTorch compilation framework automatically and efficiently.

Design Principles

- Implemented within PyTorch2's compilation infrastructure.
- Decides which kernel sequences should be captured into CUDA Graphs based on profiling and runtime characteristics.

Optimizations Introduced by PyGraph

1. **CUDA Graph-Aware Data Placement**
Extends Dynamo and Inductor to place tensors in memory layouts suitable for CUDA Graph replay.
2. **Parameter Indirection**
Inserts indirection tables to prevent excessive copying of kernel parameters when replaying CUDA Graphs.
3. **Selective CUDA Graphing**
Includes a profiler that determines whether CUDA Graph execution is beneficial for a specific workload or layer.

5 Analysis of GPU Dialect IR

5.1 Optimizations Proposed on GPU Dialect

During the study of the GPU dialect IR generated for ResNet-50, several optimization opportunities were identified. These optimizations aim to improve GPU occupancy and enhance parallel execution efficiency. The key proposals are summarized below:

- **Kernel Fusion:** In the current lowering pipeline, convolution and bias addition are emitted as separate kernels. This leads to additional kernel launch overheads from the cpu side. Fusing these operations into a single GPU kernel would eliminate unnecessary launching overhead.
- **Streaming Independent Operations in the Convolution Block:** ResNet-50 convolution blocks contain multiple independent operations that can be executed asynchronously. Streaming these kernels can overlap the execution of these operations, allowing better utilization of GPU pipelines and reducing execution latency.
- **Extracting More Parallelism Within Kernels:** The MLIR lowering maps the outer loop for every operation to GPU blocks. In some cases, the outer loop iterates only from 0 to 1, which limits the total number of launched blocks and thus overall parallelism. By skipping the outer loop and parallelizing inner loops directly, more GPU blocks can be launched, improving thread-level parallelism and occupancy.
- **Impact of Kernel Launch Configuration:** The observed IR launches certain kernels using a suboptimal configuration (e.g., 1D grids for multi-dimensional data). Switching to an appropriate configuration such as a 2D launch for convolution kernels can reduce index computation overhead and improve memory coalescing and execution efficiency.

The above optimizations target inefficiencies present in the GPU dialect IR and, if applied, have the potential to significantly minimize inference time and resource utilization when deploying ResNet-50 on GPUs.

Problem / Issue	MLIR Generated Kernel Config	Our Proposed Kernel Config	Outcome
One dimensional launch configuration by mlir	<code>mlirKernel<<< i, j >>> { for_loop_over_k }</code>	<code>ourKernel<<< dim3(i,j), k >>></code>	Not always possible due to reduction loops.
Not optimized as builtin cuda APIs	<code>mlirMemSetKernel<<< BC, channel >>> { two nested loops }</code>	<code>cudaMemSet()</code>	Performs better than the MLIR-generated version.
Non-nested for loop	<code>mlirKernel<<< 512, 1 >>> { }</code>	<code>ourKernel<<< 1, 512 >>></code>	Achieves more parallelism.
For batch = 1, outer loop iterates only once	<code>mlirKernel<<<1, 512>>> { for_loop_over_768 }</code>	<code>ourKernel<<<512, 768>>></code>	Performs better, but requires loops to be marked parallel.

6 Future Directions

- Incorporate fused kernels of CUTLASS/cuBLAS libraries.
- We can integrate MLIR and CUDA Graphs for reducing the kernel launch overhead.

7 References

- <https://dl.acm.org/doi/pdf/10.1145/3497776.3517770>
- <https://arxiv.org/pdf/2503.19779>
- <https://wisdomml.in/understanding-resnet-50-in-depth-architecture-skip-connections-and-advantages-over-other-networks/>
- <https://mlir.llvm.org/docs/>
- <https://docs.nvidia.com/cutlass/latest/>